Trust Me, Bro

Breaking Windows Authenticode Trust

Fagan Afandiyev · @KriyosArcane

DEF CON 34 · Red Team Village

how this started

a friend said something dumb. I had to find out if it was dumb.

the question

”I make my own signature and print it on”

ok but... does Windows check?

and who exactly is doing the checking?

what I assumed

file → [magic box] → trusted / not

nope.

it’s not one check. it’s a whole pipeline.

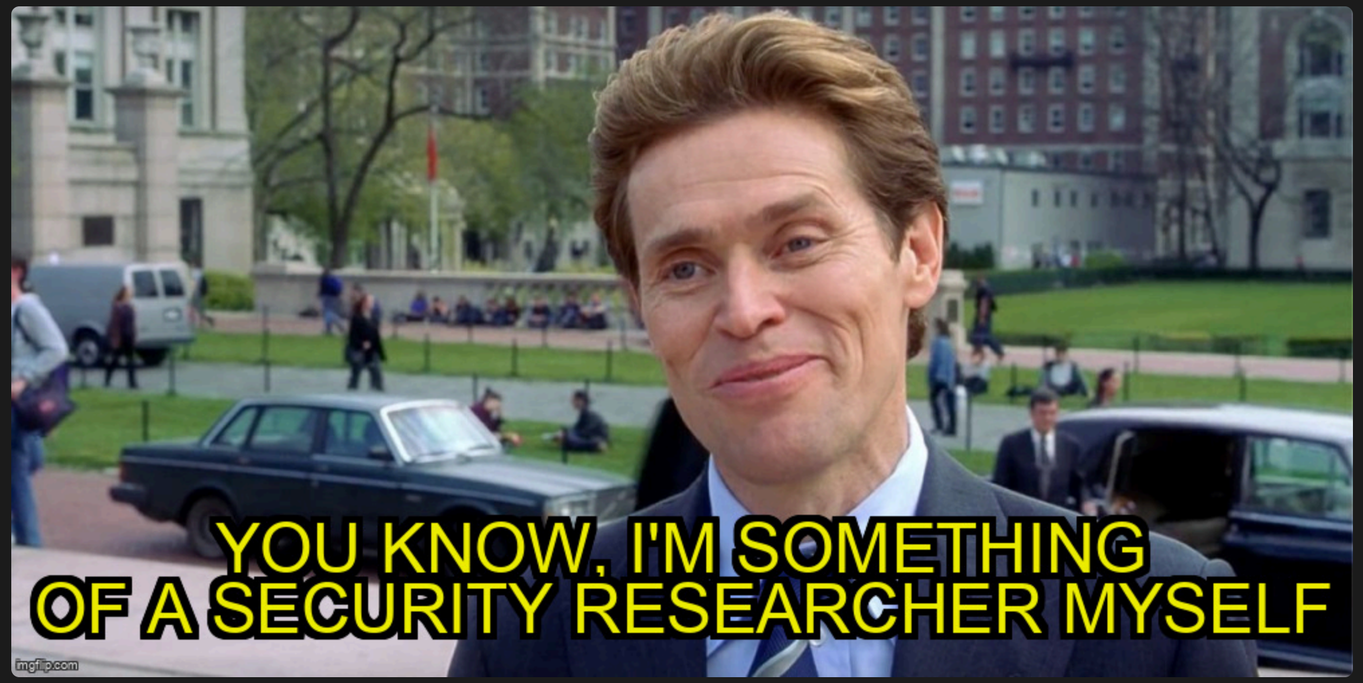
whoami

Fagan Afandiyev / KriyosArcane

Got nerd-sniped by a Discord conversation. Spent a year in the rabbit hole.

Windows trust · Code signing · Offensive tooling

USF | White Knight Labs | Microsoft (incoming)



@KriyosArcane

Authenticode Mechanics

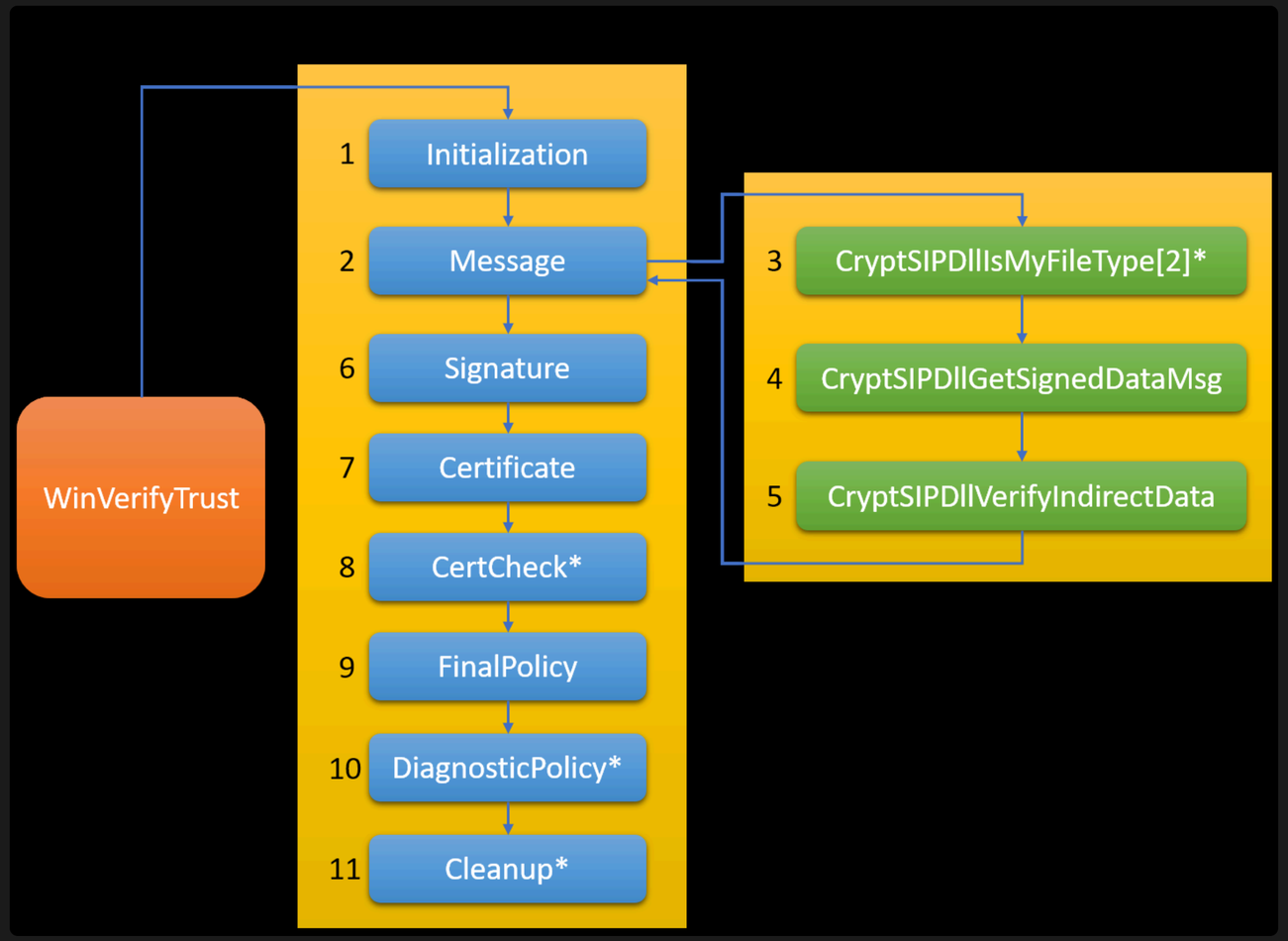
the real path

```
Caller → WinVerifyTrust → SIP → Chain → Policy → FinalPolicy → Verdict
```

each stage = a registry-configured callback

writable with local admin

WinVerifyTrust: the stages



each numbered box = a different DLL callback. green = SIP functions. (source: SpecterOps)

it’s all in the registry

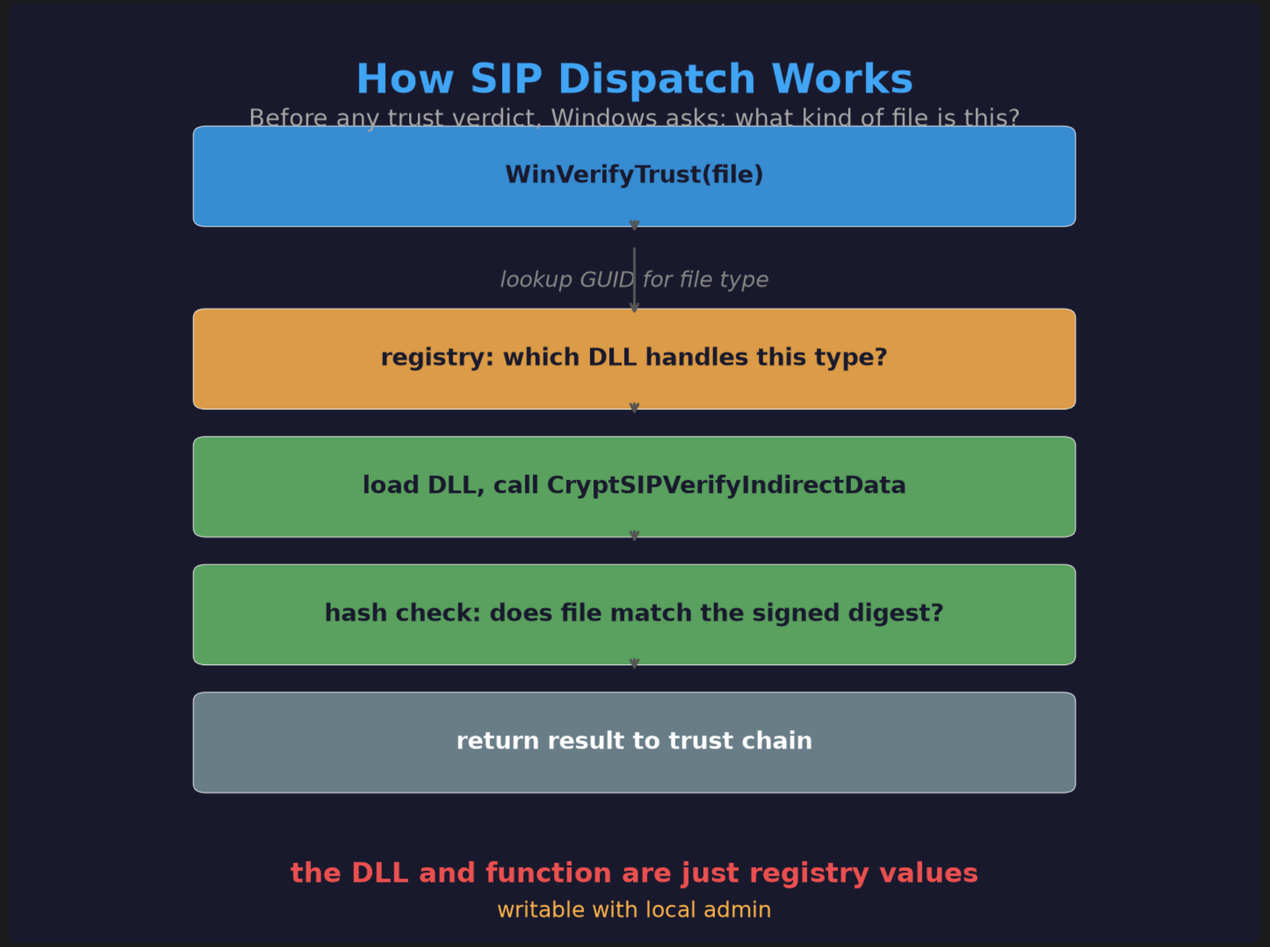
`HKLM\SOFTWARE\Microsoft\Cryptography\OID\...`

these keys say which DLL answers each trust question

writable with local admin

Finding the Seam

how trust dispatch works



SIP = Subject Interface Package. one per file type. the DLL and function are registry values.

where the signature lives

svchost.exe

Member	Offset	Size	Value	Section
Export Directory RVA	00000180	Dword	00000000	
Export Directory Size	00000184	Dword	00000000	
Import Directory RVA	00000188	Dword	0000BCCC	.rdata
Import Directory Size	0000018C	Dword	00000258	
Resource Directory RVA	00000190	Dword	00011000	.rsrc
Resource Directory Size	00000194	Dword	00000820	
Exception Directory RVA	00000198	Dword	0000F000	.pdata
Exception Directory Size	0000019C	Dword	000006D8	
Security Directory RVA	000001A0	Dword	00013000	Invalid
Security Directory Size	000001A4	Dword	000028F8	

Security Directory entry. file offset, not RVA. never mapped into memory.

steal a signature

C:\Windows\system32\cmd.exe

```
C:\> TrustMeBro.exe steal C:\Windows\System32\wintrust.dll target.exe -v

[*] Reading source file: C:\Windows\System32\wintrust.dll
[*] Certificate Table found at File Offset: 0x6f400 Size: 13808
[*] Writing to destination file: C:\Temp\freshcopy.exe
[+] Signature appended and header updated.
[+] Signature stolen from "C:\Windows\System32\wintrust.dll" to "C:\Temp\freshcopy.exe"
[*] Signature will not validate without SIP hijack or FinalPolicy.
[*] Run: C:\Users\admin\Desktop\TrustMeBro-Release\bin\TrustMeBro.exe hijack --sip-types PE  to make
 it validate.
```

copy the WIN_CERTIFICATE from a signed binary onto your target. easy.

”signed” vs verified

Explorer: ”Signed by Microsoft” ✓

PowerShell: HashMismatch ✗

having a signature ≠ passing verification

HashMismatch

file bytes don’t match the digest. only the hash check fails. everything else passes.

only the hash fails

chain ✓ cert ✓ timestamp ✓

hash ✗

so... who checks the hash? can I swap them out?

SIP Hijack

the idea

the hash verifier is a registry pointer

point it at something that returns 0

without checking anything

ntdll!DbgUiContinue

a debugger function

takes 2 params. returns 0.

CryptSIPVerifyIndirectData also takes 2 params.

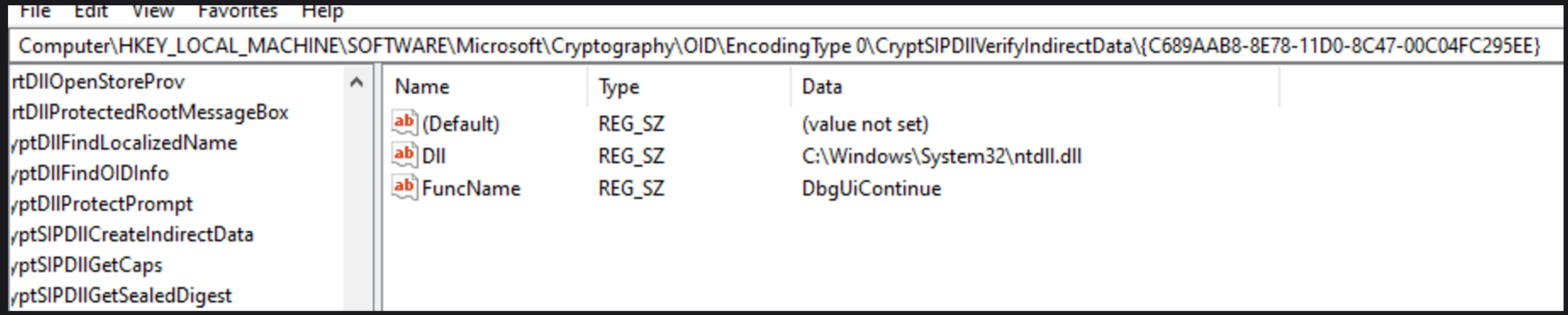
interprets 0 as ”hash is valid”

we asked a debugger about cryptography. it said yes.

normal vs hijacked

left: real verification. right: DbgUiContinue says yes to everything.

one registry value



Dll = ntdll.dll, FuncName = DbgUiContinue. that's the whole change.

same file, new answer

fresh powershell: Status = Valid. same stolen sig, same file. different answerer.

DEMO: SIP Hijack

1. Steal signature from svchost.exe
2. Verify: HashMismatch
3. TrustMeBro.exe hijack --sip-types PE
4. New process: Valid
5. TrustMeBro.exe hijack --clean
6. New process: back to HashMismatch

what this does and doesn’t do

swaps: user-mode hash verification callback

doesn’t touch: file bytes, kernel CI, real certs, catalogs

needs: local admin + fresh process

the caching gotcha

Windows caches SIP DLLs per-process

old powershell: still fails

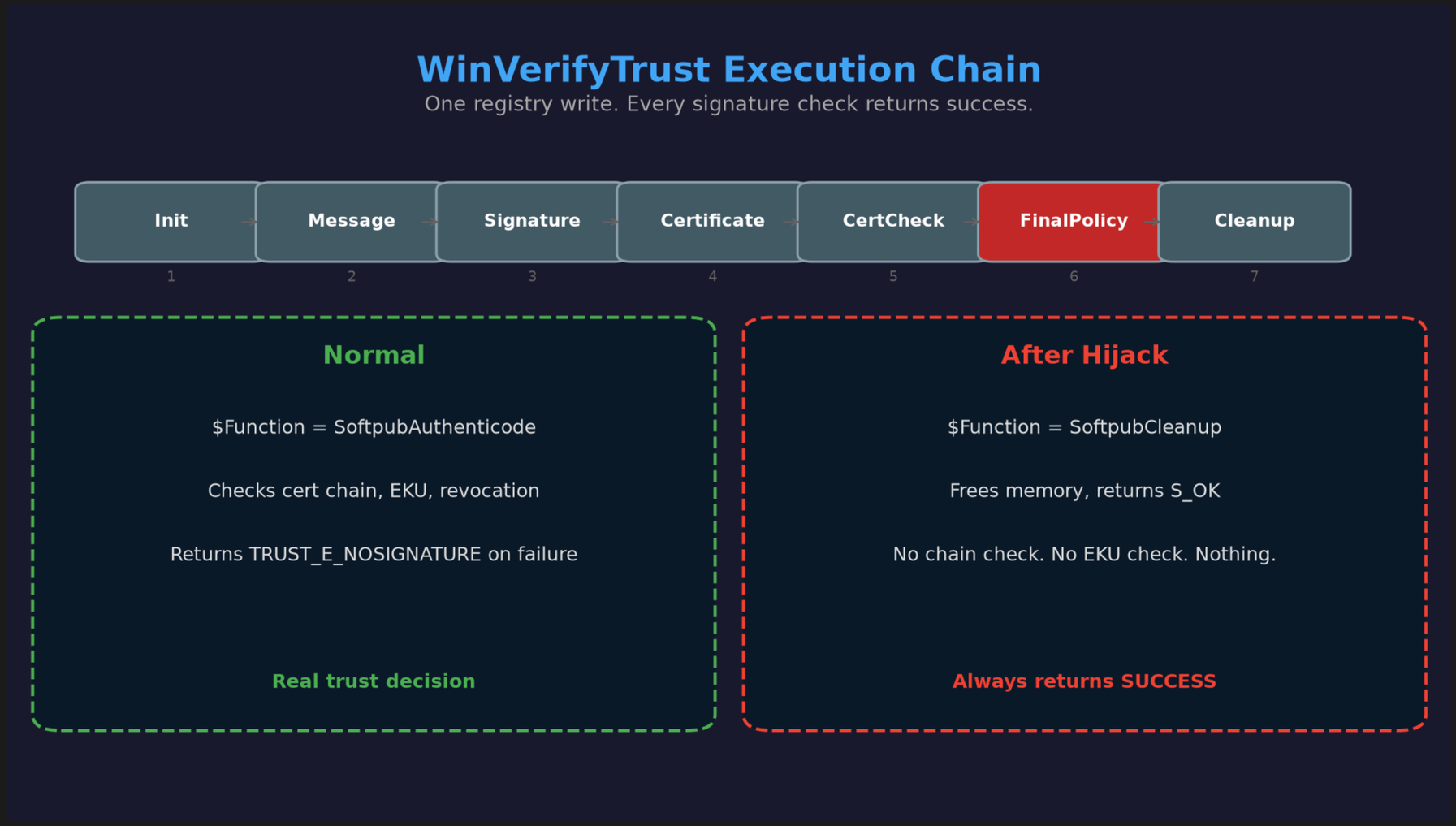
new powershell: passes

same machine, same file, different answer

FinalPolicy

SIP hijack is one file type FinalPolicyat a time. what flips everything?

FinalPolicy: the last vote



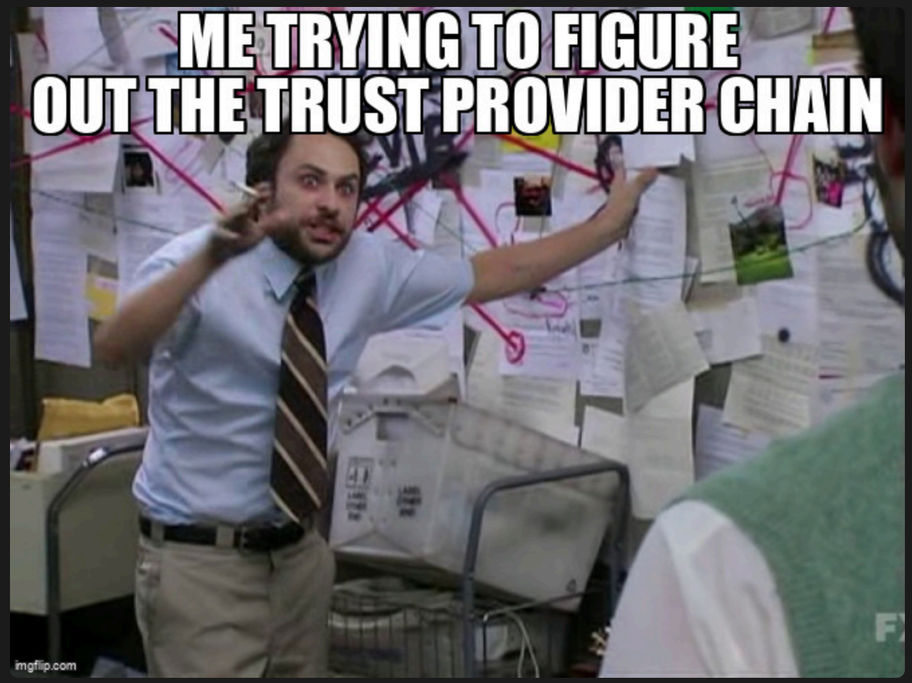
last stage before the caller gets a verdict. one function decides everything.

6 failed, 1 worked

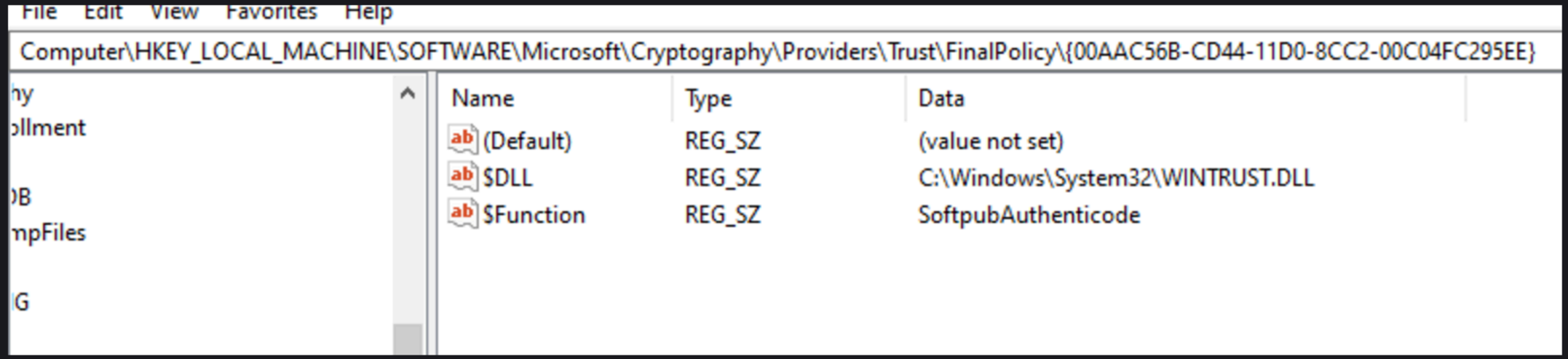
7 configurable stages

swapped each to a benign export

only FinalPolicy (stage 6) flips the verdict



SoftpubAuthenticode to SoftpubCleanup



same DLL, different export. one registry value. verdict flips.

blast radius

SIP hijack: one file type at a time

FinalPolicy: all user-mode Authenticode using this provider path

one value. trust collapses.

so we fake trust

SIP hijack: file appears validly signed ✓

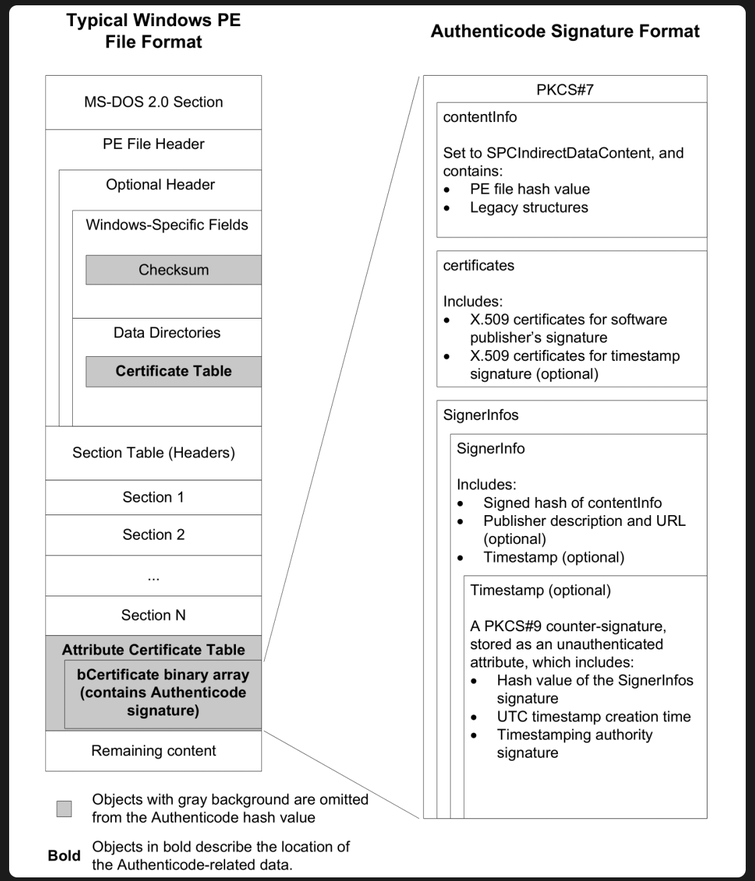
FinalPolicy: everything passes ✓

next question: hiding data inside signed files?

without breaking the signature?

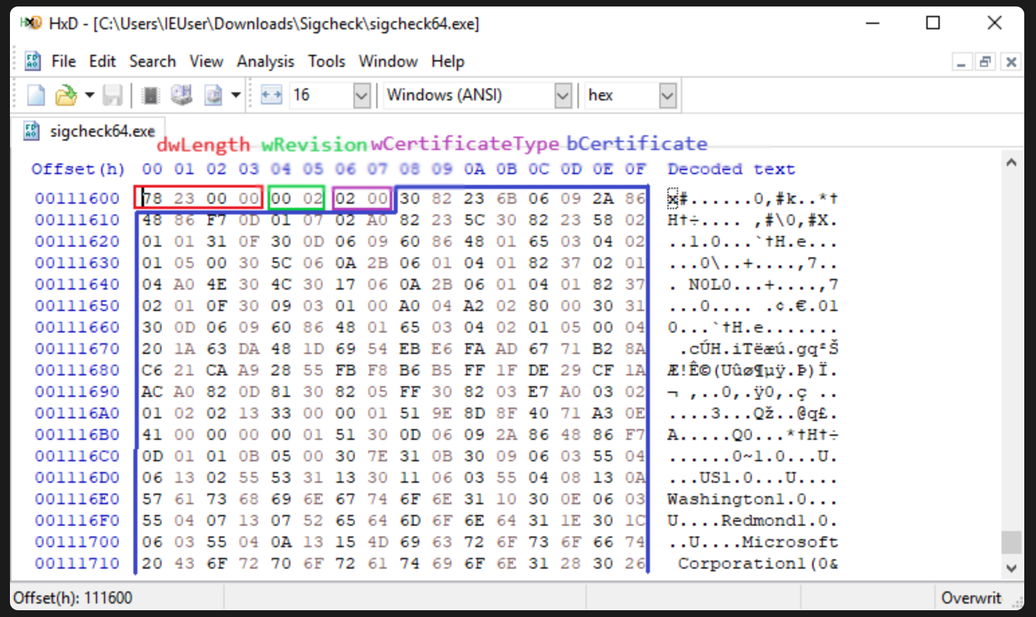
SigStash

what the hash covers



gray = skipped (checksum, cert table entry, cert table). hash covers the rested.

WIN_CERTIFICATE



dwLength, wRevision, wCertificateType, then bCertificate[] = the PKCS#7 blob.

PKCS#7 inside

PKCS#7 SignedData Structure

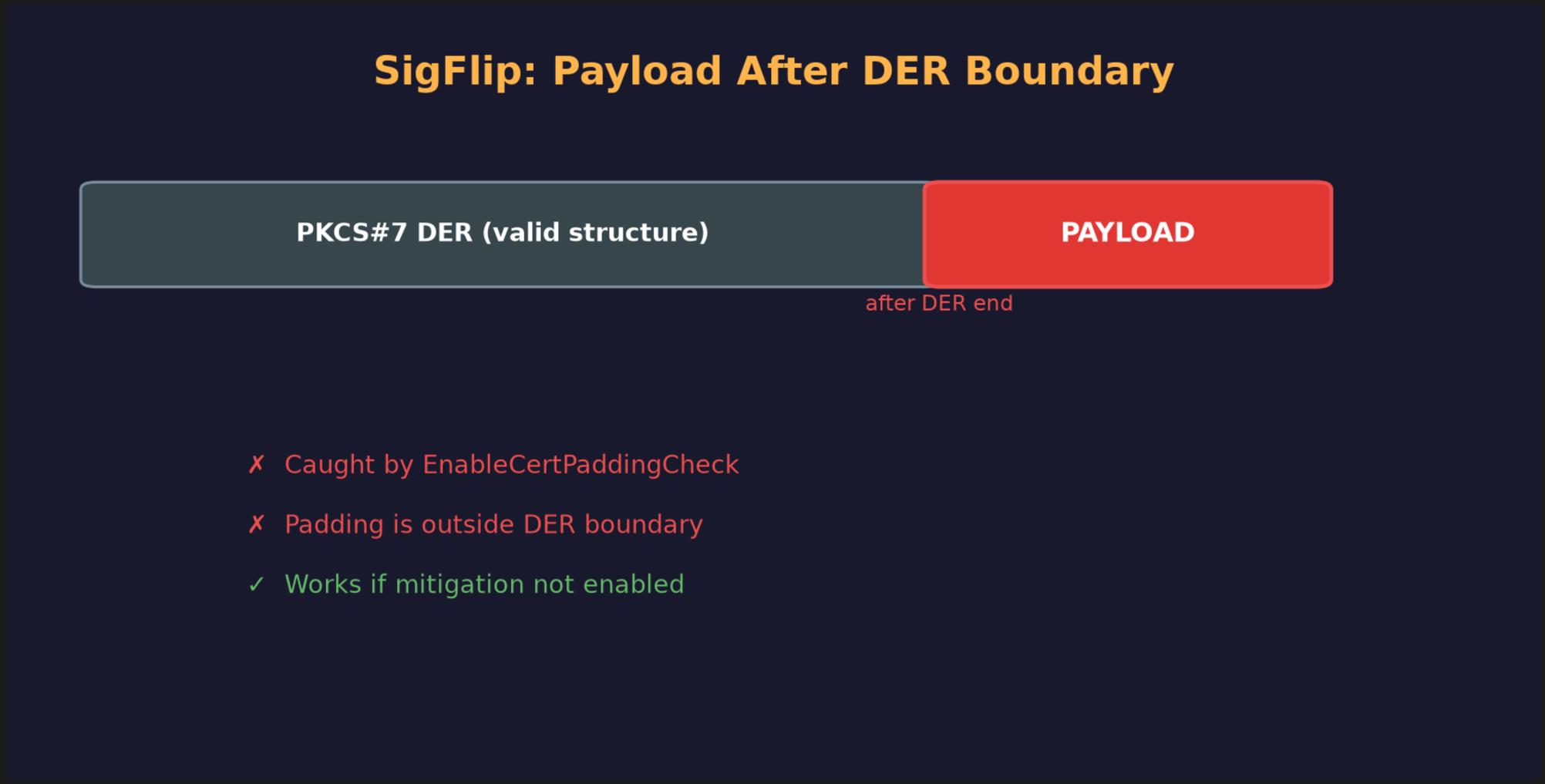
ContentInfo
└ SignedData
  └ version
  └ digestAlgorithms
  └ contentInfo (SpcIndirectData = the hash)
  └ certificates
  └ signerInfos
    └ SignerInfo
      └ signedAttrs
      └ signature (RSA/ECDSA)
      └ unsignedAttrs

Signer's signature covers these ✓

Signer's signature does NOT cover these ← PAYLOAD GOES HERE

signedAttrs = covered. unsignedAttrs = NOT covered. remember this part.

SigFlip: after the DER blob



payload after the DER boundary. EnableCertPaddingCheck kills this.

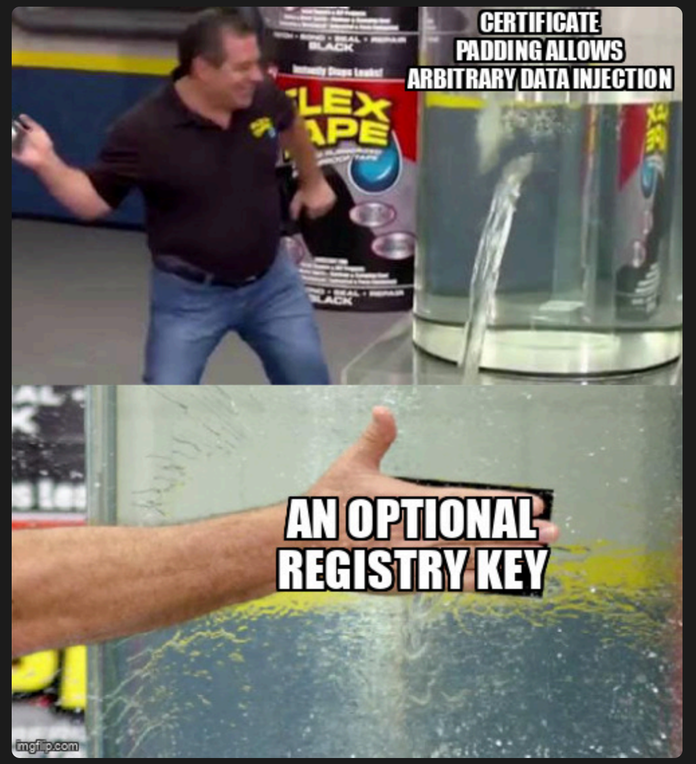
EnableCertPaddingCheck

Microsoft's mitigation for padding abuse

SigFlip: caught

SigStash: doesn't care. it's valid ASN.1.

the check looks for post-DER junk, not unsignedAttrs



what if... inside?

unsignedAttrs is not covered by the signature

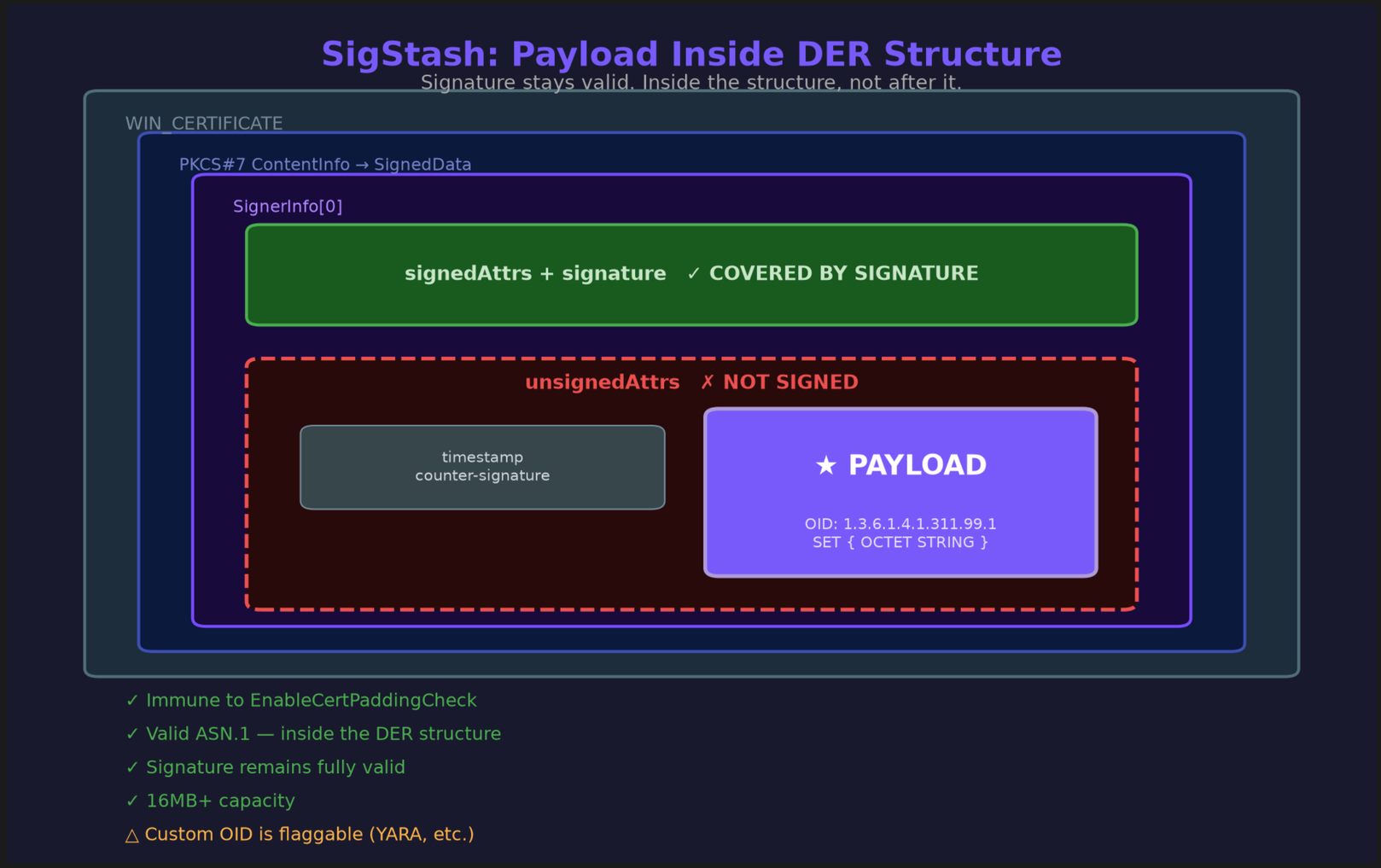
shove data in there

inside DER, not after it

signature stays valid

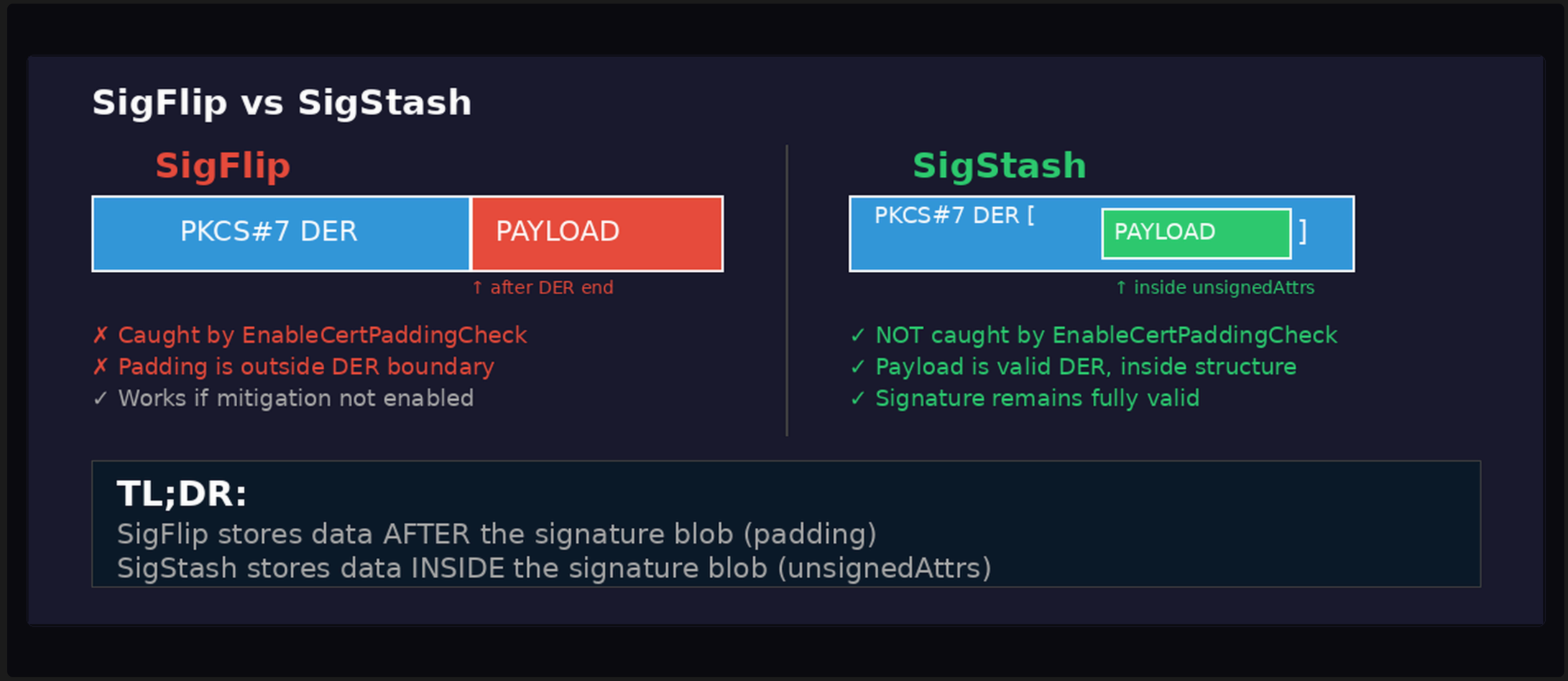
SigStash

SigStash: the unsigned pocket



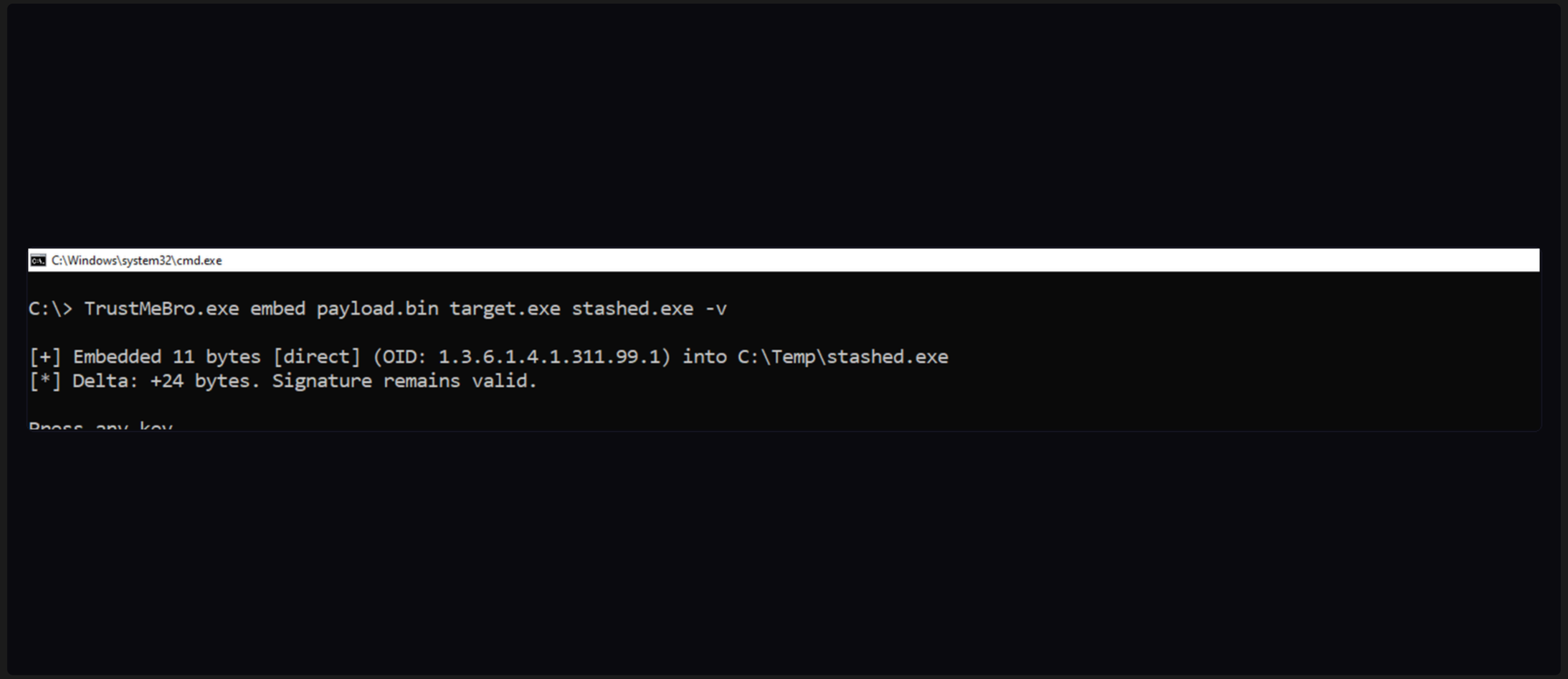
custom OID in SignerInfo.unsignedAttrs. valid ASN.1. not padding.

SigFlip vs SigStash



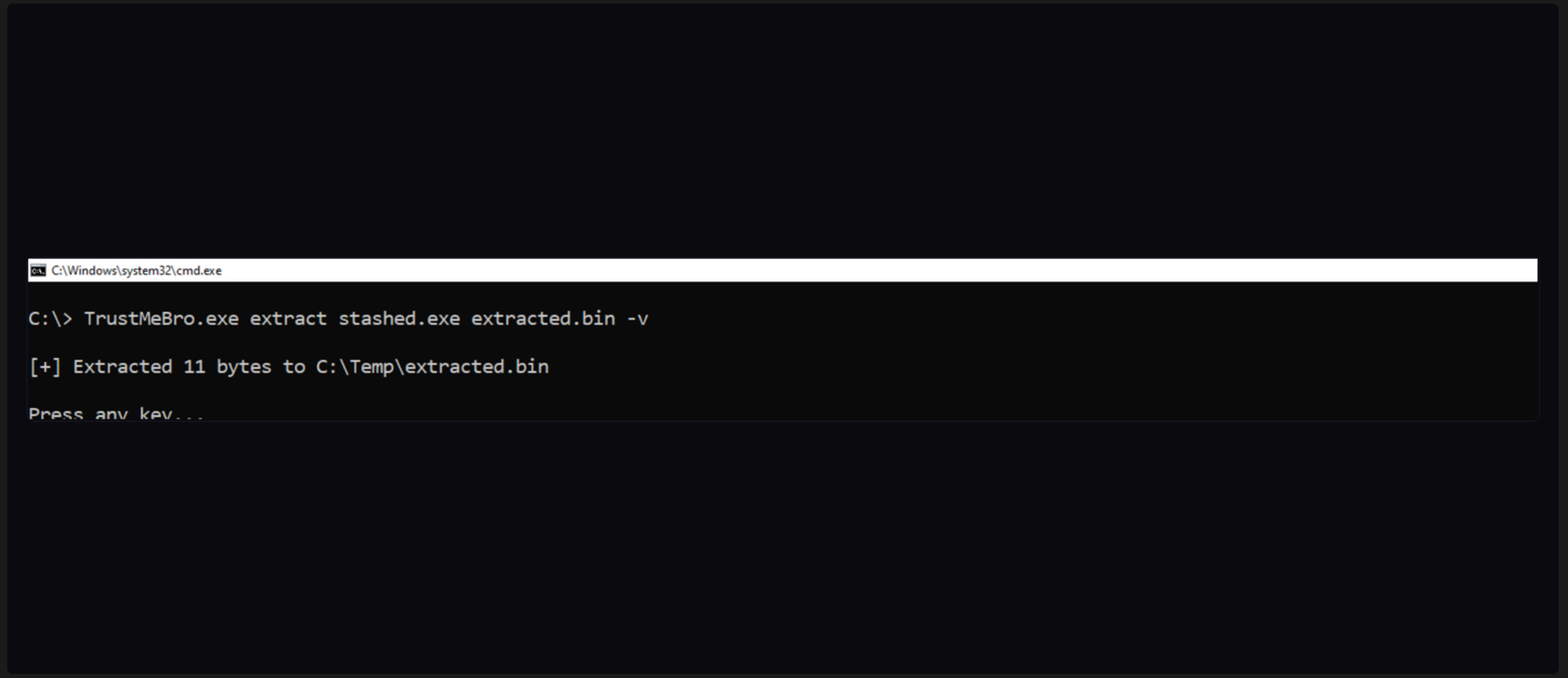
SigFlip = after DER (padding check catches it). SigStash = inside DER (unaffected).

embed



payload goes in. Get-AuthenticodeSignature still says Valid.

extract



pull it back out. SHA256 matches. round-trip clean.

Execution Surfaces

we fake trust and hide data. getting code execution from this?

SIPExec

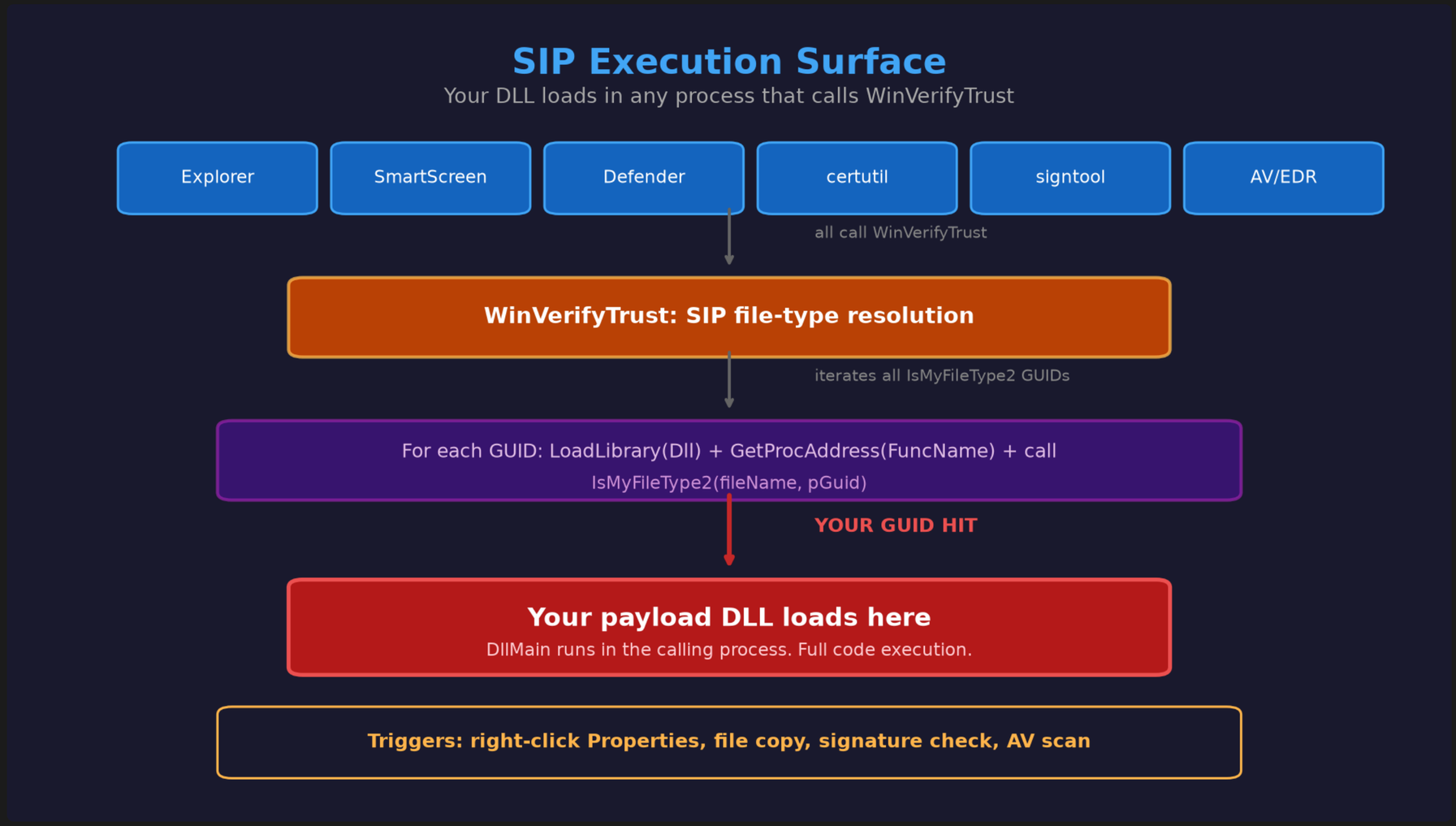
we changed which DLL loads to bypass trust

what if the DLL IS the payload?

processes that call WinVerifyTrust on your GUID will LoadLibrary your DLL

code execution triggered by a trust question

SIPExec: lateral movement



remote reg write → DCOM trigger → msiexec → SIP DLL loads → callback. no payload on disk.

DEMO: SIPExec against Elastic

1. Elastic agent running, policy: Detect + Prevent
2. Remote registry write from Linux
3. DCOM MMC20 trigger → msiexec
4. SIP DLL loads inside msiexec.exe
5. Named pipe callback → whoami
6. Zero Elastic alerts. Callback lands.

FormatGhost

SIPExec needs a remote trigger (DCOM, msiexec)

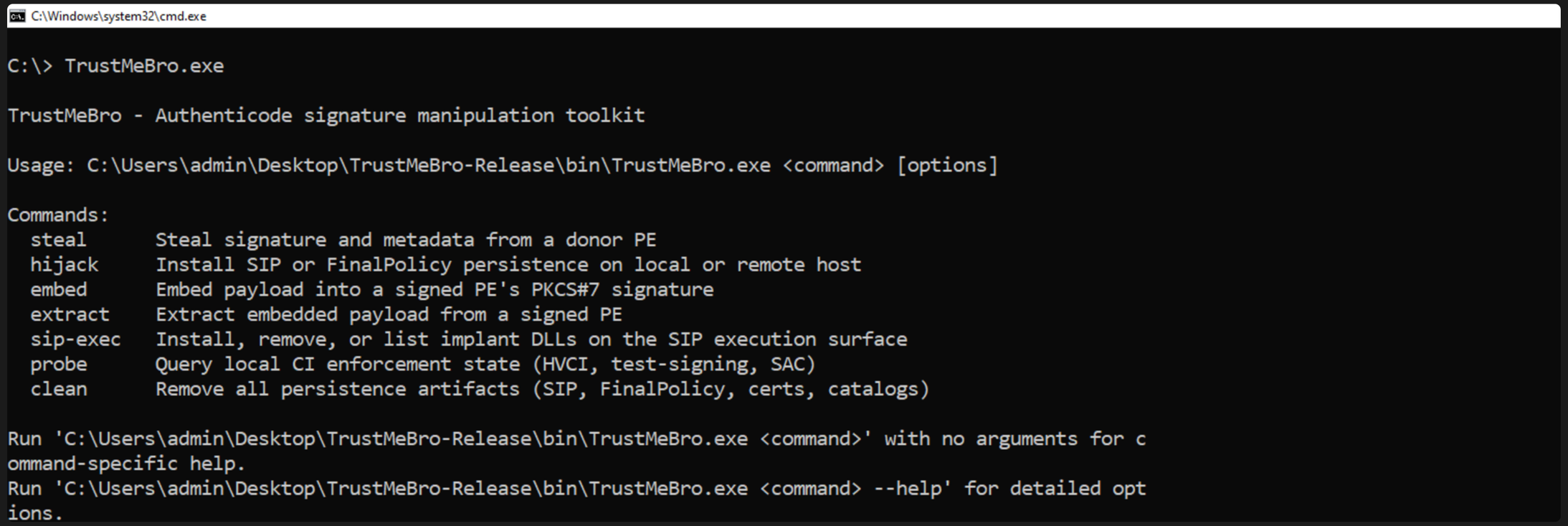
FormatGhost? right-click → Properties

the Digital Signatures tab triggers trust evaluation

looking at a file runs your code

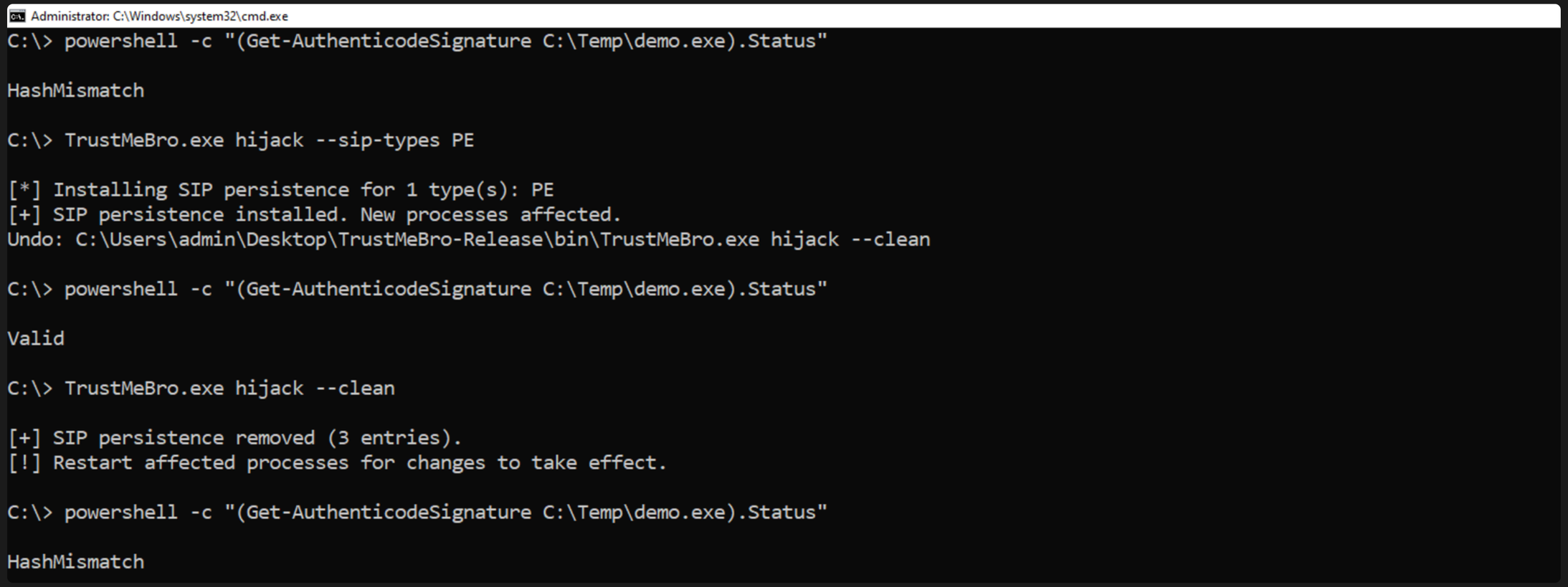
TrustMeBro

TrustMeBro: one tool, separate primitives



steal · hijack · embed · extract · sip-exec · probe · clean

One clean workflow



Inspect → apply one primitive → prove result → clean. Always clean up.

What TrustMeBro does NOT magically defeat

No forged private key

No kernel Code Integrity bypass

No universal WDAC / SAC / EDR bypass claim

No remote effect without the needed access + trigger

The real behavior is strange enough without inflating it.

Defender Guidance

detection surface

these attacks don’t modify the file. artifacts live in registry + process state.

prevent: monitor SIP + FinalPolicy registry keys. WDAC policies.

detect: registry change events. unusual DLL loads in Explorer, PowerShell, msiexec.

validate: cross-check with multiple tools. verify both x64 and WOW64 views.

Closing

Three things worth keeping

1. Signature present ≠ trust verdict.
2. Trust is a call chain. Parts are configurable.
3. Check the file, the registry, and the process.

What that first binary taught me

Trust is not a wall.

It’s questions, asked by registered trust providers and SIPs,

routed through the registry.

You change who gets asked.

Closing

Q&A

github.com/KriyosArcane/TrustMeBro

@KriyosArcane

github.com/KriyosArcane/TrustMeBro

@KriyosArcane